

Learning from Click-Logs: a Two-Stage News Recommender for EB-NeRD and MIND

Design note, CS4.406 Information Retrieval & Extraction, Assignment 2

Naman Singhal (2024114013) Yash More (2024114004)

September 2026

Summary

We built a two-stage news recommender for EB-NeRD and MIND: cheap retrieval narrows the catalogue to 200 candidates, then a LightGBM LambdaRank model re-ranks them using 22 features computed from the click-log, all restricted to data from strictly before the impression being predicted.

- **The behavioural axis is what matters.** On EB-NeRD the re-ranker scores 0.7908 test AUC against 0.5380 for retrieval alone, a gain of +0.2528 [+0.2514, +0.2543]. Popularity carries most of it: a model given popularity and nothing else already reaches 0.7348. Because that signal needs no history, cold-start and warm users gain equally (0.7910 and 0.7908), but the re-ranker is *worse* than retrieval on the most popular articles.
- **The same architecture is worth far less on MIND**, +0.0583 [+0.0565, +0.0603] test, because five of the 22 features have no source there. That contrast is a measured property of the data, not a failure to reproduce.
- **We reproduced NRMS and beat it** with one change, a longer user history: +0.0094 [+0.0086, +0.0102] test AUC. Our own re-ranker is about 0.19 AUC above it, because NRMS never sees the click log's popularity signal.
- **Honesty is nearly free.** Features unavailable at serving time would add only +0.0226 [+0.0212, +0.0240], against the +0.2528 the behavioural axis earns legitimately. We found and fixed a future-click leak that had contaminated 99.8% of validation impressions; fixing it *raised* test accuracy.
- **Serving fits the budget, and exact search is what breaks at scale.** End to end a single request takes 42.5 ms at the median and 77.3 ms at p99, inside a 100 ms target. At 10× the catalogue the exact embedding search grows 26.8×, while every other stage is roughly flat.
- **What we upload is not what we report**, and we say so: Codabench withholds the click labels, so the submitted model drops five popularity features and scores 0.7351 against 0.7908 on our own test split.

1 What we built

The task is news recommendation from click-logs. The unit of everything is an *impression*: at time t a user is shown a list of *candidate* articles and clicks some of them. Our job is to order each impression's candidates so the clicked ones come first. We extend our two Assignment-1 retrieval systems (lexical and semantic) with the *behavioural* signal in the logs: what users clicked, how often each article has been clicked or shown, and how fresh it is.

1.1 Data and evaluation protocol

Splits are by time, never random. A random split would let the model learn from Thursday to predict Wednesday, which no deployed system can do. EB-NeRD val begins 23 May 07:00 and test

dataset	articles	train / val / test impressions	leaderboard test set	
			impressions	articles
EB-NeRD small (Danish)	20,738	168,522 / 64,365 / 244,647	13,536,710	125,500
MIND small (English)	65,238	95,071 / 61,894 / 73,152	2,370,727	120,961

Table 1: Development data and the held-out leaderboard sets. Splits are temporal: every train impression precedes every val impression, which precedes every test impression.

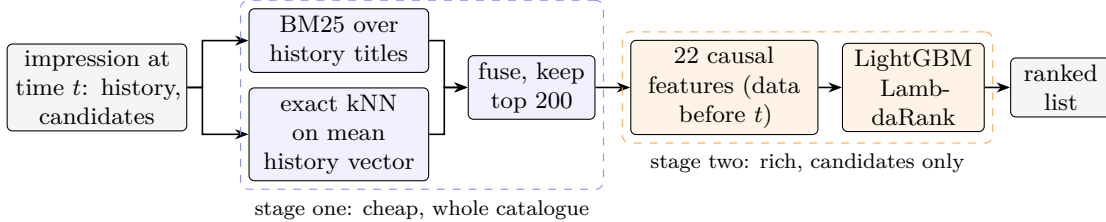


Figure 1: The two-stage pipeline. Stage one touches every article, so it must stay cheap per article. Stage two touches only the candidates, so it can afford per-candidate features.

25 May 07:00; MIND uses the last day of its training log as val and its dev log as test.

Metric: per-impression AUC, computed inside each impression and averaged over those with both a click and a non-click. We never pool candidates across impressions, because the ranking is only judged inside one: pooling rewards features that merely differ between impressions, and the effect is large, with article freshness reading 0.5861 pooled against 0.5087 per impression.

Significance: paired bootstrap 95% intervals [10], 1,000 resamples, both systems scored on the *same* resampled impressions so their correlated errors cancel. A gain counts only if its interval excludes zero. Selection happens on val; test is scored once.

1.2 Architecture: retrieve, then re-rank

Stage one, candidate generation (carried from Assignment 1). *BM25* [4, 5] is a lexical score: the query is the titles of the user’s last N clicks ($N = 30$ on EB-NeRD, 100 on MIND), and an article scores highly when it shares rare words with them. Stemming is on for both languages and a Danish stopwords list is applied. The *semantic* retriever represents the user as the mean of their clicked articles’ vectors and finds the nearest articles by cosine with an exact FAISS `IndexFlatIP` [6]. Vectors are the provided 768-dimensional `contrastive_vector` for EB-NeRD and 384-dimensional `all-MiniLM-L6-v2` [7] for MIND, both chosen by measurement in Assignment 1. The two scores are standardised within each candidate pool and mixed linearly, with the weight chosen on val. Each retriever keeps its top 200 candidates. Both indexes are held in RAM.

Stage two, the re-ranker. A LightGBM [8] gradient-boosted tree model with the *LambdaRank* objective [9]. LambdaRank learns from pairs of candidates inside the same impression and weights each pair by how much swapping them would change nDCG, so training optimises the ranking itself rather than a click probability. We measured this choice rather than assuming it (Table 4). Settings: learning rate 0.05, 31 leaves, at least 50 rows per leaf, 90% feature and row subsampling, up to 400 rounds with early stopping on val nDCG, seed 13.

1.3 Features and the behaviour-window boundary

Every feature for an impression at time t uses only data from strictly before t . We call this the *behaviour-window boundary*. Click and exposure counts are computed by binary search over each article’s sorted event times with a strict “ $< t$ ” cut, so an event at exactly t is excluded.

family	features and what they measure	EB-NeRD	MIND
stage-one scores	bm25 , emb (the two retrieval scores); emb_rank (the embedding score’s rank inside the impression)	yes	yes
click popularity	pop_causal : clicks before t in a 6 h window (unbounded on MIND); pop_24h : the same over 24 h; pop_velocity : their ratio, “rising now” versus “steadily popular”; pop_rank , pop_rel_max : standing among this impression’s candidates	yes	yes
exposure	the same five shapes (exp_causal , exp_24h , exp_velocity , exp_rank , exp_rel_max) counting how often an article was <i>shown</i> before t	yes	yes
user match	cat_match : share of the user’s history in the candidate’s category; ent_overlap : shared named entities with the history	yes	yes
context	hist_len : history length; n_cands : number of candidates in the impression	yes	yes
freshness	age_hours , recency (24 h half-life decay of age)	yes	no
engagement	user_read , user_scroll : mean read time and scroll depth of past clicks; engage_sim : cosine to a read-time-weighted user vector	yes	no
total		22	17

Table 2: The re-ranker’s features. Rank and relative features exist because the metric is decided inside one impression: a popularity of 200 dominates a quiet impression and is ordinary in a busy one.

Tests assert the boundary on every split, and each test was checked by injecting a future click and confirming it fails (Section 2.5).

Five features cannot exist on MIND, and we drop them explicitly. MIND’s publication time is null for all 65,238 articles, and it ships no read-time or scroll data at all. A column of zeros looks like a real feature to LightGBM, so the builder refuses to emit any feature that is all-null or constant. This matches the data audit we did before designing features: EB-NeRD is rich in behavioural signal, MIND has little beyond the click itself.

Exposure (“how often was this article shown before t ”) was added for a practical reason and kept for a better one. The leaderboard test files withhold the clicks, so click popularity cannot be computed there (Section 4), but the lists of shown articles are present. A live recommender also knows its own exposure log immediately, while click feedback arrives later, so exposure is arguably the more realistic signal.

2 Results

Unless stated otherwise, numbers are per-impression AUC on EB-NeRD small and MIND small, with paired bootstrap 95% intervals, selected on val and scored once on test.

2.1 Before and after re-ranking

Two baselines appear in this report and they are not the same thing. Here “stage one” is the fused retrieval score ranking the candidates directly (0.5380 on test). The ablation grid in Table 4 instead compares against **stage1_only**, a ranker trained on the two retrieval scores alone, which is slightly stronger at 0.5430. That is why the same result reads +0.2528 here and +0.2223 there.

On EB-NeRD the re-ranker is worth +0.2528 AUC over fused retrieval, [+0.2514, +0.2543] on test, and +0.2357 [+0.2329, +0.2385] on val. The val and test intervals overlap, so the gain generalises across the time boundary rather than being an artefact of what we selected on. Every

system	AUC	MRR	MRR, all clicks	nDCG@5	nDCG@10
<i>EB-NeRD small, test</i>					
BM25 (stage one)	0.5107 [0.5094, 0.5120]	0.3257 [0.3245, 0.3269]	0.3253 [0.3241, 0.3264]	0.3577 [0.3563, 0.3591]	0.4409 [0.4397, 0.4420]
embeddings (stage one)	0.5397 [0.5384, 0.5409]	0.3492 [0.3480, 0.3503]	0.3487 [0.3475, 0.3498]	0.3831 [0.3818, 0.3845]	0.4627 [0.4616, 0.4638]
fused (stage one)	0.5380 [0.5367, 0.5392]	0.3474 [0.3463, 0.3486]	0.3470 [0.3458, 0.3481]	0.3813 [0.3800, 0.3827]	0.4613 [0.4602, 0.4624]
two-stage re-ranker	0.7908 [0.7898, 0.7918]	0.5685 [0.5671, 0.5699]	0.5678 [0.5665, 0.5693]	0.6330 [0.6318, 0.6344]	0.6650 [0.6639, 0.6662]
fused + lifetime popularity*	0.5797 [0.5785, 0.5810]	0.3688 [0.3677, 0.3700]	0.3684 [0.3672, 0.3696]	0.4101 [0.4088, 0.4115]	0.4841 [0.4830, 0.4852]
<i>MIND small, test</i>					
BM25 (stage one)	0.5685 [0.5663, 0.5706]	0.3108 [0.3085, 0.3133]	0.2693 [0.2673, 0.2717]	0.2868 [0.2844, 0.2894]	0.3479 [0.3456, 0.3504]
embeddings (stage one)	0.6369 [0.6347, 0.6390]	0.3510 [0.3486, 0.3535]	0.3061 [0.3039, 0.3085]	0.3341 [0.3316, 0.3367]	0.3937 [0.3913, 0.3961]
fused (stage one)	0.6381 [0.6358, 0.6402]	0.3536 [0.3511, 0.3561]	0.3075 [0.3053, 0.3098]	0.3361 [0.3335, 0.3388]	0.3956 [0.3934, 0.3981]
two-stage re-ranker	0.6964 [0.6943, 0.6983]	0.4027 [0.4001, 0.4054]	0.3459 [0.3436, 0.3483]	0.3854 [0.3827, 0.3882]	0.4458 [0.4435, 0.4483]

Table 3: All accuracy metrics on the test split, with 95% bootstrap intervals beneath each mean. “Stage one” rows are the retrieval scores ranking the same candidates; the re-ranker reorders them. *Lifetime popularity is not available at serving time and is reported only to price what it would buy (Section 2.5). Validation-split figures are in Appendix F.

other accuracy metric moves the same way, so this is a genuine reordering rather than a trade between metrics.

On MIND the same architecture is worth +0.0583 [+0.0565, +0.0603], far less. Two things compound. Five of the 22 features have no source in MIND, including every freshness and engagement signal, so the behavioural axis is thinner. And MIND’s stage one is already stronger in absolute terms (0.6381 against EB-NeRD’s 0.5380), which leaves less headroom to begin with. We report the contrast rather than presenting one system that is quietly two: a smaller gain on a dataset with less signal is a measured property of the data.

One family of features did transfer, and it was the one built under duress. An earlier round of seven features moved MIND by nothing significant at all. The five exposure features, added because Codabench withholds the click labels, are worth +0.0297 [+0.0278, +0.0316] on MIND test and +0.0458 [+0.0440, +0.0478] on val. How often an article was *shown* is apparently a signal both corpora carry, where freshness and engagement are specific to EB-NeRD.

2.2 What each design choice was worth

Every choice below was made by measuring the alternative, one change at a time, with a paired bootstrap interval. Rows marked “10 features” were measured before the feature set grew; the two feature rows are the incremental steps to the shipped EB-NeRD model.

Three of these deserve a sentence. **The objective matters:** with a binary objective the model’s most-used features become absolute freshness, while LambdaRank leans on popularity contrasts inside the impression, which is what decides the ranking. **Recent popularity beats lifetime popularity:** counting only the last 6 hours asks “how popular is this right now”, the right question for news. In isolation the window moved the feature from 0.6902 to 0.7349 AUC, but inside the ranker the gain is far smaller because other features partly cover for it, so we quote the ranker

choice	compared against	val Δ [95% CI]	test Δ [95% CI]	verdict
re-ranking at all	stage one alone	+0.2138 [+0.2110, +0.2168]	+0.2223 [+0.2209, +0.2238]	core result
LambdaRank objective (10 features)	binary objective	+0.0161 [+0.0150, +0.0173]	+0.0165 [+0.0159, +0.0171]	kept
6 h click-popularity window (10 features)	all past clicks	+0.0034 [+0.0026, +0.0043]	+0.0062 [+0.0058, +0.0067]	shipped
seven rank, velocity and context features	the 10 features	+0.0113 [+0.0104, +0.0122]	+0.0130 [+0.0124, +0.0134]	shipped
five exposure features	the 17 features	+0.0249 [+0.0238, +0.0261]	+0.0255 [+0.0249, +0.0261]	shipped
read-time-weighted user vector	uniform vector only	+0.0003 [−0.0003, +0.0010]	+0.0005 [+0.0002, +0.0008]	kept, tiny
linear fusion of BM25 and embeddings	embeddings alone	+0.0022 [+0.0010, +0.0034]	−0.0017 [−0.0023, −0.0010]	does not generalise
title-only BM25 documents	title and abstract	+0.0025 [+0.0002, +0.0047]	+0.0043 [+0.0031, +0.0055]	not shipped [†]
stemming off, MIND	stemming on	−0.0023 [−0.0034, −0.0013]	−0.0028 [−0.0039, −0.0018]	keep on
stemming off, EB-NeRD	stemming on	−0.0003 [−0.0019, +0.0014]	−0.0002 [−0.0011, +0.0007]	keep on, for cost

Table 4: Per-impression AUC change from each choice, on EB-NeRD unless marked. The first row is the `stage1_only` arm of the 17-feature grid. [†]Title-only documents also cost val recall@200 by −0.0024 [−0.0035, −0.0013], and recall is what stage two consumes.

figure. **Fusion helps on val and hurts on test**, both significantly. The mixing weight chosen on val does not transfer across the time boundary, which is moot next to stage two but is an honest negative result.

2.3 Baseline reproduced, then beaten (EB-NeRD)

The baseline. NRMS [3] is a neural recommender with two parts: a news encoder that turns each article into a vector, and a user encoder that uses attention over the user’s clicked articles to decide which past clicks matter for the current prediction. We ran the organisers’ `ebnerd_nrms_docvec.py` unmodified on EB-NeRD small with the provided document vectors, history size 20, 4 negatives per positive, batch 32 and seed 16. It reached val AUC **0.6484** (best epoch 3, early-stopped at 7, 54 minutes on 16 CPU cores of a cluster node).

That number is not comparable to ours. The benchmark script trains on the organisers’ train and validation blocks together and validates on its own last day, which in our split means it trains on our val and test impressions. So we also drove the same model, settings, vectors and data loader from our temporal split: train on our train, stop early on our val, score our test once. That run is the baseline we compare against.

The one principled change: a longer history, 20 to 50 clicks. The motivation is our own measurement. On clean data, the mean-of-history user vector improves steadily as the history grows, from 0.5416 at 10 clicks to 0.5545 at 100 (Appendix E). NRMS builds its user representation from exactly that history, so it should benefit too. It does: the gain is significant on both splits and larger on test than on val, the opposite of what over-fitting to val would produce. We stopped at 50 because attention cost grows with the square of the history length. A guard asserts every article id reaches the model (coverage 1.0000 on all four checks), since a silent id mismatch would train on

EB-NeRD small, our temporal split	val AUC	test AUC
NRMS, benchmark script and protocol (not comparable)	0.6484	–
NRMS baseline, history size 20	0.5969	0.5938
NRMS improved, history size 50	0.6042	0.6032
improved – baseline	+0.0072 [+0.0058, +0.0088]	+0.0094 [+0.0086, +0.0102]
our two-stage re-ranker	0.7885	0.7908

Table 5: NRMS reproduced and beaten with one change, paired bootstrap. Both NRMS arms score the identical 64,365 val and 244,647 test impressions, aligned by impression id.

slice	impressions	fused (stage one)	two-stage re-ranker	gain	fused + lifetime pop.*
<i>EB-NeRD small, test</i> cold: history ≤ 34 clicks; head: clicked article with ≥ 349 train clicks					
cold	25,105	0.5449 [0.5407, 0.5488]	0.7910 [0.7879, 0.7942]	+0.2461	0.5906 [0.5865, 0.5945]
warm	219,542	0.5372 [0.5358, 0.5386]	0.7908 [0.7897, 0.7917]	+0.2535	0.5785 [0.5771, 0.5798]
head	803	0.6353 [0.6143, 0.6574]	0.5530 [0.5316, 0.5728]	–0.0823	0.6921 [0.6700, 0.7129]
tail	243,844	0.5377 [0.5364, 0.5390]	0.7916 [0.7906, 0.7925]	+0.2539	0.5793 [0.5781, 0.5806]
<i>MIND small, test</i> cold: history ≤ 3 clicks; head: clicked article with ≥ 152 train clicks					
cold	7,529	0.5601 [0.5533, 0.5676]	0.6480 [0.6415, 0.6548]	+0.0880	–
warm	65,623	0.6470 [0.6448, 0.6492]	0.7019 [0.6999, 0.7039]	+0.0549	–
zero_history	2,214	0.5125 [0.5007, 0.5246]	0.6288 [0.6161, 0.6404]	+0.1163	–
head	860	0.6347 [0.6167, 0.6508]	0.7884 [0.7783, 0.7984]	+0.1538	–
tail	72,292	0.6381 [0.6359, 0.6401]	0.6953 [0.6932, 0.6975]	+0.0571	–

Table 6: Per-impression AUC by slice, test split, with 95% intervals. “Gain” is the re-ranker minus fused retrieval. *Lifetime popularity, not servable, shown for comparison.

zero vectors and still report a believable AUC.

Why our system beats NRMS by about 0.19 AUC. Not because trees beat attention: NRMS reads article content and click order only, and has no click popularity, our most valuable signal. Our own content-only arm, `stage1_only`, scores 0.5498 val, close to NRMS. The comparison is between a model that sees the click log’s behavioural signal and one that does not.

2.4 Slices and beyond-accuracy

An average can hide a system that fails a subgroup, so we split the test impressions by how much history the user has and by how often the clicked article was clicked in training. **Slice sizes are printed because the thresholds are not obvious:** over 90% of articles in both datasets have zero training clicks, so a percentile over the whole catalogue would be zero and would put every impression in the head slice. Taking it over articles seen in training gives a head threshold of 349 clicks on EB-NeRD and leaves head at 803 impressions, 0.3% of the split.

The gain is uniform across the slices that matter, with cold, warm and tail all improving by about 0.25 AUC. **Cold and warm users are now indistinguishable** (0.7910 and 0.7908), which is a property rather than an achievement: the model’s strongest features are popularity and

EB-NeRD small, val, 64,365 impressions	per-impression AUC
causal features only (servable)	0.7523
plus lifetime totals (not servable)	0.7749
difference, paired bootstrap	+0.0226 [+0.0212, +0.0240]

Table 7: Metrics with and without serving-unavailable features, measured on the 10-feature model with the 6 h window.

exposure, which need no history, so having little history costs almost nothing.

Head articles are the exception, and the result reverses across the split boundary. On validation the re-ranker beats fused retrieval on head (0.7080 [0.6905, 0.7253] against 0.6638 [0.6467, 0.6800]); on test it loses (0.5530 [0.5316, 0.5728] against 0.6353 [0.6143, 0.6574]). Both differences have non-overlapping intervals, but a significant effect that flips sign across a temporal boundary is a property of the window, not a stable claim. **So we cannot claim the re-ranker helps on the most popular articles, and on test it is clearly worse there than retrieval alone.** One untested explanation fits: our popularity features count only the last six hours and are rank-normalised inside the impression, a deliberate “trending now” signal, and head articles are the steadily popular ones it discounts. Consistent with that, the leaky arm with lifetime popularity is the best system on head (0.6921), the only slice where it wins, and MIND, whose popularity window is unbounded, goes the other way: 0.7884 against 0.6347 on its 860 head impressions, its largest slice gain. MIND’s users also gain most where they have least, 0.6288 against 0.5125 on the 2,214 impressions with no history at all.

The accuracy gain is not paid for in diversity or novelty (Appendix C). The re-ranker has the highest novelty of any system on both datasets and diversity close to BM25’s, so it is not collapsing onto a few popular stories. It does cost a little catalogue coverage: 0.2051 against 0.2026 on EB-NeRD test and 0.0529 against 0.0494 on MIND, roughly 3 to 7% fewer distinct articles ever surfacing. That is the real trade and it is small.

Coverage is quoted without an interval, on purpose. It counts *distinct* articles rather than averaging over impressions, and a bootstrap draws only about 63% of the impressions, so the articles only the missing ones would have surfaced simply vanish and every resample undercounts. Our own attempt makes the bias visible: coverage on EB-NeRD test is 0.2067 for BM25 while the resampled spread is [0.1875, 0.1920], an “interval” that excludes the value it describes. We report the point estimate and keep the spread as a diagnostic.

2.5 Anti-gaming: serving-unavailable features and the leak we found

What the system declines to use. EB-NeRD ships three lifetime totals per article (views, page views, read time) summed over the whole collection period, so for any impression they include the future and cannot exist when serving. We trained the same ranker on the same rows with and without them. The difference is small next to what the honest system already earns:

In Assignment 1 the same totals were worth +0.042 to +0.075, more than our whole semantic axis, because they were compared against systems with no popularity signal at all. Here they compete with click counts restricted to the past, which capture most of the same information legitimately. Cheating would buy +0.0226, while the behavioural axis is worth +0.2528 over stage one, roughly an order of magnitude more. Both arms of this comparison are the 10-feature model, so it prices the leak against that round rather than against the shipped 22. End to end, the servable re-ranker (0.7908 on test) far outscores stage one with lifetime popularity added (0.5797). MIND cannot run this comparison at all: its lifetime columns are null for all 65,238 articles.

index	size	RAM (MiB)	disk (MiB)	RAM / disk
BM25 (sparse matrix)	297,145 postings, 30,388 terms	7.0	3.0	2.32×
FAISS flat index	20,738 × 768 float32	60.8	60.8	1.00×
feature store	5,514,689 candidate rows	547.0	240.4	2.28×

Table 8: Footprints measured separately in memory and serialised, since the two differ by different factors per structure.

Enforcement. Two tests guard the boundary. One asserts that no click, exposure or engagement value used for an impression comes from at or after its timestamp, on every split; the other fails if any scoring module reads a lifetime column. Each was verified by injecting a violation and watching it fail, after we found checks that passed without checking anything (Section 5).

The first recounts popularity against the **6 h window the shipped feature actually uses**, with a *two-sided* mask over the click times and a different code path from the builder’s binary search. Two-sided is the load-bearing part: asserting only “no more than the clicks before t ” is satisfied by a window reaching past t so long as it stays narrow, so the weaker check passes on the leak it exists to catch. We confirmed the stronger one is live by mutation, not by argument: changing the window to 12h or to unbounded without rebuilding makes both boundary tests fail. The suite is **29 passed, 6 skipped, 0 failed**.

The leak we found, and what fixing it did. EB-NeRD ships its history in two blocks, each describing clicks before *that block* begins. Our feature builder merged both, letting the later block overwrite the earlier for 11,658 users. Our val split is carved from the earlier block, so val impressions were described by history from their own future: **99.8% of validation and 99.9% of training impressions were affected**, while test was clean at 0.0% because it comes from the later block itself (Appendix A).

We found it by chasing a metric discrepancy, not with a test: two implementations of the same user vector scored 0.6473 and 0.5506, and making one reproduce both numbers with only the history source changed confirmed the cause, a +0.0967 inflation on that feature. The fix reads one block per split, and a new test that fails on injection covers it. Two consequences show the leak was real and the fix was right. The read-time-weighted user vector’s apparent +0.0085 contribution fell to +0.0002, not significant, so its entire value had been the leak. And test AUC *rose* from 0.7429 to 0.7461, because test features were always clean and only the model, now trained without future information, changed.

3 Serving and scale

Every number here came from one machine, a laptop with 8 physical (12 logical) cores, 15.3 GB RAM and no GPU, timing the shipped 22-feature model with its 399 trees. Latencies from different machines are not comparable, so none are mixed. Requests are timed **one at a time**, never batched: batching raises throughput while making each request slower, and it is the tail that a user notices.

3.1 Index memory, in RAM and on disk

The dense index is the largest structure despite covering the fewest objects: a vector spends memory on every dimension of every article, while an inverted index spends it only on terms that occur. It is also the only one whose RAM and disk sizes agree, because a flat index *is* its raw matrix, with nothing to compress or reconstruct. Both facts matter for the scaling result.

stage	p50 (ms)	p95 (ms)	p99 (ms)	share of p50
tokenise query	1.49	2.36	5.31	4%
BM25 top 200	3.82	9.36	12.41	9%
embedding top 200 (exact)	18.65	35.54	48.08	44%
assemble features	0.60	0.71	2.10	1%
LightGBM predict	16.37	26.85	34.72	39%
total	42.45	64.66	77.31	

Table 9: End-to-end latency over 2,000 single requests on EB-NeRD small, stage by stage.

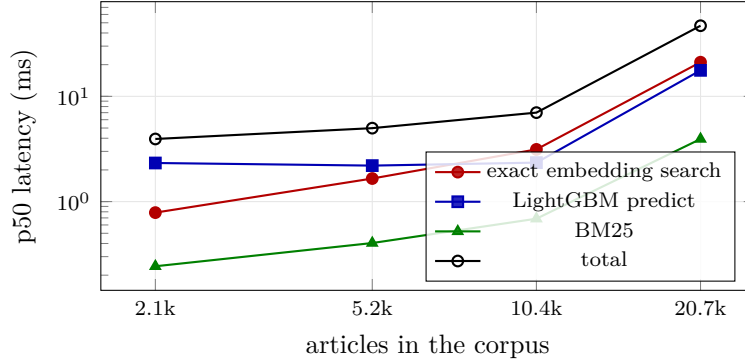


Figure 2: Per-stage median latency against corpus size, both axes logarithmic. The figures behind it are in Appendix D.

3.2 Latency of a single request

The median request takes 42.5 ms and the 99th percentile 77.3 ms. Exact embedding search is 44% of the median and the re-ranker 39%, so the two expensive stages are the dense scan and stage two. Reporting per stage is what makes this actionable: it names the component to attack.

The accuracy gain was not free. The same bench on the earlier 10-feature model with 311 trees gave a median of 22.06 ms and a p99 of 64.90 ms, so going to 22 features roughly doubled the median. That is an honest cost of the +0.2528 AUC, and it only shows up when serving is measured rather than assumed.

3.3 Cost per 1000 queries at a target SLA

Against a target of p99 under 100 ms, the measured p99 of 77.3 ms **meets the SLA, with 1.3× headroom** rather than the 1.5× the smaller model had. One core sustains 24 queries per second, measured. Scaling to 8 physical cores gives 188 QPS, a *projection* assuming requests are independent and scale linearly. At an assumed \$0.040 per vCPU-hour that is **\$0.000472 per 1000 queries**, also projected: the price is an assumption and only the latency it multiplies is measured. We count physical cores because this path is dense floating-point work, where a second thread on the same core shares an execution unit and adds much less than a real core.

3.4 What breaks first at 10×

Rather than extrapolate one measurement under an assumed growth law, which is the assumption in question, we subsampled the corpus and re-timed every stage.

Exact embedding search breaks first. Over a 10.0× corpus it grows 26.8×, where a flat scan should be at worst linear. The cause is the memory hierarchy, not arithmetic: at the smallest corpus the vectors occupy 6.1 MiB and sit in cache, and at full size they occupy 60.8 MiB and every query

streams them from RAM. **At 10× the catalogue this is a memory-bandwidth problem, not a FLOPs problem**, so buying cores would not fix it.

The re-ranker also slows down, and it is not doing more work. Its cost grew 7.6×, which is surprising because it scores only the candidate pool, whose size is set by the retrievers rather than by the corpus. We measured the pool instead of assuming it: across the same 10.0× corpus the mean pool goes from 345 to 388 candidates, a factor of 1.13. The work really is flat, so the extra time is the same work executing more slowly. Every stage slows at the largest corpus, including feature assembly, which touches only that same pool. The consistent reading is that the dense scan’s memory traffic evicts everything else from cache, so the index does not merely cost its own stage time, it taxes the whole request.

Two growth figures are upper bounds, and we can now say by how much. Subsampling the corpus also shortens the query, because it is built from the titles of history articles and a removed article contributes nothing. Measured on the same runs: mean query length falls from 144.6 tokens at full corpus to 17.7 at a tenth, and resolvable history items from 29.1 to 3.6. So the 4.6× for tokenising and 16.2× for BM25 mix corpus size with query length and overstate the corpus effect. Exact search is unaffected, since a flat scan costs vectors times dimensions whatever the query holds, which is why the verdict rests on it.

This overturns our Assignment-1 conclusion, which was that the per-impression Python loop would break before the linear algebra. The lever that matters now is replacing exact search with an approximate index, which Assignment 1 measured as 10 to 20× faster and set aside as unnecessary at the time, flagging a revisit at 10×. This is that revisit and the answer is the opposite. On this evidence it would speed up more than its own stage, since it is also what is taxing the others. What it would cost in recall at this catalogue size we have not measured, so the replacement is motivated but unquantified.

Repeated runs on this machine differ by roughly $\pm 20\%$, so these timings should never be compared against numbers from another machine.

4 Leaderboard submissions

The uploaded system is a re-ranker, but not the one reported above, and the gap is not in our favour. Both are stated so that the better number never stands in for the uploaded one.

Why the two differ. The Codabench test files withhold `article_ids_clicked`. We checked this by comparing columns, not by assumption: the training behaviours file minus the test behaviours file leaves exactly the outcome columns (clicks and the next read and scroll values). Our five click-popularity features count clicks before each impression, and on the test period those clicks *are* the withheld labels, so the features cannot be computed there at any cost. We replaced them with the five exposure features, which only need the in-view lists the test files do contain. We also dropped `bm25` from the submission for cost rather than availability. It scores every article for every distinct history, which projects to about 1.6 hours on MIND-large, and it is worth only +0.0004 test AUC in the ablation.

Scale and validation. The EB-NeRD test set has 13,536,710 impressions and 205,925,868 candidate pairs; MIND-large has 2,370,727. Neither fits in memory, so prediction streams in slices. Scoring EB-NeRD took 3 h 06 m at 44 GB peak on a cluster node, after three earlier implementations were abandoned once their measured rates projected 38, 90 and 135 hours; batching the LightGBM call, 13.5M invocations down to 541, was the fix that mattered. Each rewrite was checked for byte-identical output. Because MIND allows one submission per day, every file passes an offline

EB-NeRD small, our test split	features	per-impression AUC
reported system	22	0.7908
uploaded system (no click popularity, no <code>bm25</code>)	16	0.7351
uploaded – reported, paired bootstrap		−0.0556 [−0.0565, −0.0547]

Table 10: The gap between what we report and what we upload, measured on our own labelled test split. The MIND upload uses 11 features and scores 0.6460 on our MIND test split.

leaderboard	AUC	MRR	nDCG@5	nDCG@10
MIND (Codabench 13967)	[pending]	[pending]	[pending]	[pending]
EB-NeRD (Codabench 2469)	[pending]	[pending]	[pending]	[pending]

Table 11: Official leaderboard scores. [To fill in after upload; screenshots are in Appendix H.]

validator first: exact inner filename (`prediction.txt` for MIND, `predictions.txt` for EB-NeRD), nothing else in the zip, row count, row order, and each row a permutation of $1..n$. Row order matters most, since ranks are positional and a reordered file scores as a plausible bad model rather than failing. The validator was itself checked by injecting five violations and confirming each one fails.

Reading the official MIND MRR. MIND’s scorer averages the reciprocal rank over every clicked article, while the common definition credits only the first. The two agree when an impression has one click. On our MIND test split, where 28.8% of impressions have more than one click, the definitions give 0.3108 against 0.2693 for the same ranking, with AUC and nDCG unchanged. Our harness reports both, and the “all clicks” column is the one to compare with the leaderboard.

5 Observations that generalise

One metric name can hide two definitions. This happened three times. Our retrieval stage printed a *hit rate* (share of impressions with at least one clicked article retrieved) under the label *recall* (share of clicked articles retrieved); on MIND test they read 0.0333 and 0.0226 and reconcile exactly, $0.0333 \times 0.6775 = 0.0226$. MIND’s official MRR averages over all clicks while ours credited the first, 0.2693 against 0.3108 for one ranking. And two implementations of “the user vector” read different history files, which is how we found the leak in Section 2.5. In each case the number reproduced on one dataset and not the other, because the definitions only diverge when impressions have several clicks (28.8% on MIND, 0.5% on EB-NeRD). When a number fails on exactly one dataset, check the definition first.

A feature’s isolated score is not its contribution. Impression size, `n_cands`, is the same for every candidate in an impression, so on its own it scores exactly 0.5000, and yet removing it costs −0.0066 test AUC, the second largest loss in the grid. It helps only through interactions, by telling the model how crowded the impression is. The reverse also happens: freshness has high LightGBM split gain but removing it changes almost nothing, because popularity already carries most of the same information. We therefore quote ablation deltas, never importance.

Leave-one-out deltas do not add up. In the 17-feature grid, removing `pop_causal` alone or `user_read` alone makes the model slightly better on val. Removing both is significantly *worse* on both splits (−0.0005 val, −0.0007 test), and keeping only the features with significant positive deltas loses 0.0240 test AUC. Correlated features cover for each other, so a single-feature ablation answers “what does this add given the rest” and not “what should we keep”. We ship all features.

An underpowered result is not a null result. Our stage-one ablations were first run on the $10\times$ smaller EB-NeRD demo set, where two results had intervals spanning zero. At full `ebnerd_small`

scale they resolved in opposite directions: dropping the abstract from BM25 went from +0.0009 [−0.0062, +0.0081] to a real +0.0025 [+0.0002, +0.0047], while Danish stemming stayed flat within ± 0.0004 at intervals tight enough to exclude the +22 to +36% recall gain Assignment 1 had claimed. We now re-run any interval that spans zero at larger scale before recording a verdict.

A check must test the artifact, not a proxy for it. We found **six** checks that passed without checking anything: a MIND leakage test that filtered every row away (a null list is not an empty list), an anti-gaming guard that scanned zero files after a directory move, an install check that trusted an exit code while the import was broken, a job monitor that matched its own command line, leakage tests that never looked at the engagement arrays the model reads, and an injection test *deleted by its own name*, two test functions in one file sharing a name so the second replaced the first and the popularity injection check ran zero times from the day it was written. Reading the file cannot catch that; counting collected tests can, and 12 collected against 15 written is what exposed it. Every test must now fail on an injected violation before it counts, and how many actually run is part of what we check.

A guard that fires on prose teaches you to delete the prose. The same anti-gaming test searched source as raw text and failed on `src/rerank/q9.py`, whose only mention of the lifetime columns is a comment recording that MIND has none. The obvious fix, allow-listing the file, would have passed the suite while removing the guard from a module that never read them. It now parses the code, so a comment costs nothing and `pl.col("total_inviews")` still fails.

6 Limitations and open questions

- **Parts of Q1 are not modelled.** We have no within-session features (EB-NeRD’s session id is unused) and no explicit position-bias correction. Exponential time decay on the history was neutral to harmful at every setting in Assignment 1 and was not revisited inside the ranker.
- **The baseline covers one dataset.** NRMS was reproduced and beaten on EB-NeRD only; a MIND baseline was not reproduced. The history length of 100 that our sweep favours was not tried in NRMS, whose cost grows quadratically with it.
- **The final model may be under-trained.** The 22-feature model’s best iteration is 399 of a 400-round cap, so it was still improving when training stopped; its reported gain is more likely understated than inflated.
- **Approximate search is motivated but unmeasured.** Section 3 shows the exact index is what breaks at $10\times$, but we did not measure the recall an IVF or HNSW index would cost at our catalogue size.
- **MIND gains little from behaviour.** With five features missing and a stronger stage one, the MIND re-ranker’s gain is a small fraction of EB-NeRD’s. This is a property of the data, but it means our strongest claims are about EB-NeRD.
- **One stage-one improvement is unshipped.** Title-only BM25 documents gain ranking AUC on both datasets but cost EB-NeRD val recall@200 (−0.0024), which is exactly what stage two consumes.
- **The static anti-gaming scan has a blind spot we have not closed.** It inspects parsed code rather than raw text, so it sees `pl.col("total_inviews")` but not a column name assembled at runtime from parts. Closing it needs a runtime check on what the scorer actually reads, which we did not build.

References

- [1] J. Kruse, K. Lindskow, S. Kalloori, M. Polignano, C. Pomo, A. Srivastava, A. Uppal, M. R. Andersen, and J. Frellsen. EB-NeRD: a large-scale dataset for news recommendation. In *Proceedings of the ACM RecSys Challenge 2024*, 2024.
- [2] F. Wu, Y. Qiao, J.-H. Chen, C. Wu, T. Qi, J. Lian, D. Liu, X. Xie, J. Gao, W. Wu, and M. Zhou. MIND: a large-scale dataset for news recommendation. In *Proceedings of ACL*, pages 3597–3606, 2020.
- [3] C. Wu, F. Wu, S. Ge, T. Qi, Y. Huang, and X. Xie. Neural news recommendation with multi-head self-attention. In *Proceedings of EMNLP-IJCNLP*, pages 6389–6394, 2019.
- [4] S. Robertson and H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [5] X. H. Lù. BM25S: orders of magnitude faster lexical search via eager sparse scoring. arXiv:2407.03618, 2024.
- [6] J. Johnson, M. Douze, and H. Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Knowledge and Data Engineering*, 2019.
- [7] N. Reimers and I. Gurevych. Sentence-BERT: sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP*, 2019.
- [8] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. LightGBM: a highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, 2017.
- [9] C. J. C. Burges. From RankNet to LambdaRank to LambdaMART: an overview. Technical Report MSR-TR-2010-82, Microsoft Research, 2010.
- [10] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

A The engagement leak, measured

split	impressions	matched to the later history block	history at or after the impression
train	168,522	144,087	143,998 (99.9%)
val	64,365	61,026	60,901 (99.8%)
test	244,647	244,647	0 (0.0%)

Table 12: Future-click contamination in the engagement features before the fix (Section 2.5). Test was clean because it is drawn from the later block itself.

B Full ablation grids

Each arm removes one feature and retrains the same model; **stage1_only** keeps only the two retrieval scores and **pop_only** only click popularity. A positive “full – arm” means the removed feature helped. Bold marks intervals excluding zero. These grids were run before the exposure features were added (17 features on EB-NeRD, 12 on MIND), which is why their full-model AUC sits below the shipped system’s.

arm	val AUC	full – arm, val [95% CI]	full – arm, test [95% CI]
full model	0.7636		test AUC 0.7653
stage1_only	0.5498	+0.2138 [+0.2110, +0.2168]	+0.2223 [+0.2209, +0.2238]
pop_only	0.7348	+0.0288 [+0.0275, +0.0301]	+0.0360 [+0.0352, +0.0367]
minus_cat_match	0.7574	+0.0062 [+0.0053, +0.0071]	+0.0069 [+0.0064, +0.0074]
minus_n_cands	0.7580	+0.0056 [+0.0050, +0.0063]	+0.0066 [+0.0062, +0.0069]
minus_bm25	0.7625	+0.0011 [+0.0006, +0.0017]	+0.0004 [+0.0001, +0.0007]
minus_pop_rank	0.7629	+0.0007 [+0.0000, +0.0013]	+0.0003 [+0.0000, +0.0006]
minus_hist_len	0.7631	+0.0005 [+0.0000, +0.0010]	+0.0005 [+0.0002, +0.0007]
minus_pop_velocity	0.7632	+0.0004 [−0.0001, +0.0008]	−0.0002 [−0.0004, +0.0001]
minus_user_scroll	0.7632	+0.0003 [−0.0001, +0.0008]	−0.0003 [−0.0006, −0.0001]
minus_age_hours	0.7632	+0.0003 [−0.0001, +0.0008]	+0.0004 [+0.0002, +0.0006]
minus_engage_sim	0.7633	+0.0003 [−0.0003, +0.0010]	+0.0005 [+0.0002, +0.0008]
minus_emb	0.7637	−0.0001 [−0.0006, +0.0004]	−0.0003 [−0.0006, −0.0001]
minus_emb_rank	0.7637	−0.0001 [−0.0006, +0.0003]	−0.0001 [−0.0003, +0.0002]
minus_pop_24h	0.7638	−0.0003 [−0.0008, +0.0003]	−0.0001 [−0.0004, +0.0002]
minus_ent_overlap	0.7639	−0.0003 [−0.0008, +0.0002]	−0.0006 [−0.0009, −0.0004]
minus_pop_rel_max	0.7639	−0.0003 [−0.0009, +0.0003]	+0.0002 [−0.0002, +0.0005]
minus_recency	0.7639	−0.0004 [−0.0008, +0.0001]	−0.0005 [−0.0008, −0.0003]
minus_user_read	0.7641	−0.0005 [−0.0010, −0.0000]	−0.0006 [−0.0008, −0.0003]
minus_pop_causal	0.7644	−0.0008 [−0.0013, −0.0003]	−0.0006 [−0.0009, −0.0003]

Table 13: EB-NeRD small, 17-feature model, 64,365 val and 244,647 test impressions.

arm	val AUC	full – arm, val [95% CI]	full – arm, test [95% CI]
full model	0.6539		test AUC 0.6666
pop_only	0.5475	+0.1064 [+0.1040, +0.1089]	+0.0957 [+0.0935, +0.0978]
stage1_only	0.6372	+0.0167 [+0.0147, +0.0187]	+0.0293 [+0.0274, +0.0313]
minus_emb	0.6497	+0.0042 [+0.0033, +0.0052]	+0.0010 [+0.0001, +0.0018]
minus_cat_match	0.6502	+0.0037 [+0.0028, +0.0047]	+0.0070 [+0.0062, +0.0079]
minus_bm25	0.6517	+0.0022 [+0.0013, +0.0031]	+0.0027 [+0.0020, +0.0035]
minus_hist_len	0.6530	+0.0008 [+0.0002, +0.0016]	−0.0001 [−0.0007, +0.0006]
minus_pop_causal	0.6531	+0.0008 [+0.0001, +0.0014]	−0.0009 [−0.0016, −0.0003]
minus_pop_velocity	0.6534	+0.0005 [−0.0003, +0.0013]	+0.0000 [−0.0007, +0.0007]
minus_n_cands	0.6537	+0.0001 [−0.0007, +0.0011]	+0.0009 [+0.0002, +0.0017]
minus_ent_overlap	0.6538	+0.0001 [−0.0006, +0.0007]	−0.0005 [−0.0012, +0.0001]
minus_pop_rank	0.6540	−0.0001 [−0.0008, +0.0006]	+0.0003 [−0.0004, +0.0010]
minus_pop_rel_max	0.6541	−0.0002 [−0.0010, +0.0007]	+0.0001 [−0.0007, +0.0009]
minus_pop_24h	0.6542	−0.0003 [−0.0010, +0.0004]	+0.0003 [−0.0003, +0.0010]
minus_emb_rank	0.6542	−0.0003 [−0.0010, +0.0004]	−0.0007 [−0.0014, −0.0001]

Table 14: MIND small, 12-feature model, 61,894 val and 73,152 test impressions.

C Beyond-accuracy metrics

system	diversity	novelty (bits)	coverage
<i>EB-NeRD small, test</i>			
BM25 (stage one)	0.7968 [0.7961, 0.7974]	16.966 [16.964, 16.969]	0.2067
embeddings (stage one)	0.7803 [0.7796, 0.7809]	17.007 [17.004, 17.010]	0.2050
fused (stage one)	0.7820 [0.7814, 0.7826]	16.999 [16.996, 17.002]	0.2051
two-stage re-ranker	0.7901 [0.7895, 0.7907]	17.084 [17.082, 17.086]	0.2026
<i>MIND small, test</i>			
BM25 (stage one)	0.8428 [0.8416, 0.8439]	16.927 [16.924, 16.929]	0.0536
embeddings (stage one)	0.8215 [0.8202, 0.8227]	16.926 [16.923, 16.929]	0.0523
fused (stage one)	0.8182 [0.8169, 0.8195]	16.925 [16.922, 16.927]	0.0529
two-stage re-ranker	0.8008 [0.7996, 0.8021]	17.003 [17.001, 17.005]	0.0494

Table 15: Top-10 beyond-accuracy metrics, test split. Diversity is the share of top-10 pairs from different categories; novelty is mean self-information in bits, so higher is less obvious; coverage is the share of the catalogue that ever reaches a top 10. Coverage carries no interval for the reason given in Section 2.4.

D Scaling measurements

corpus	articles	FAISS MiB	BM25 p50	exact kNN p50	rerank p50	total p50	total p99
10%	2,074	6.1	0.24	0.79	2.33	3.94	7.28
25%	5,184	15.2	0.40	1.66	2.20	4.99	8.42
50%	10,369	30.4	0.69	3.13	2.35	7.00	10.22
100%	20,738	60.8	3.93	21.11	17.66	46.87	109.00

Table 16: Latency against corpus size in milliseconds, the numbers behind Figure 2. Same machine and same model throughout.

E Isolated-feature sweeps

Per-impression AUC of one feature used alone as the score, on val. An isolated score cannot be absorbed by a correlated neighbour, which makes it a clean screen, but a winner here is only a candidate: the popularity window gained +0.0447 in isolation and +0.0034 val inside the ranker.

popularity window	EB-NeRD	MIND	min. scroll depth	AUC	history kept
1 h	0.7082	0.5622	0%	0.5545	100.0%
6 h	0.7349	0.5475	10%	0.5521	88.5%
24 h	0.7254	0.5516	25%	0.5559	78.5%
72 h	0.7045	0.5506	50%	0.5590	58.4%
168 h	0.6902	0.5502	75%	0.5606	46.7%
unbounded	0.6902	0.5502	90%	0.5609	41.1%

recency half-life	1 h	3 h	6 h	12 h	24 h	48 h	96 h	168 h
EB-NeRD AUC	0.5088	0.5087	0.5087	0.5088	0.5087	0.5088	0.5088	0.5088

history clicks	1	5	10	20	50	100
uniform user vector	0.5221	0.5368	0.5416	0.5483	0.5528	0.5545
read-time weighted	0.5221	0.5362	0.5434	0.5495	0.5554	0.5570

Table 17: Top left: the click-popularity window. Top right: dropping history clicks with low scroll depth (EB-NeRD, 100 clicks). Middle: the recency half-life, flat to four decimals over a $168\times$ range. Bottom: history length and read-time weighting of the user vector on clean history.

F Validation-split evaluation

system	AUC	MRR	MRR, all clicks	nDCG@5	nDCG@10
<i>EB-NeRD small, val</i>					
BM25 (stage one)	0.5205 [0.5180, 0.5231]	0.3418 [0.3395, 0.3440]	0.3414 [0.3391, 0.3436]	0.3794 [0.3767, 0.3821]	0.4607 [0.4586, 0.4629]
embeddings (stage one)	0.5506 [0.5482, 0.5530]	0.3594 [0.3572, 0.3617]	0.3589 [0.3567, 0.3612]	0.4017 [0.3991, 0.4044]	0.4778 [0.4757, 0.4801]
fused (stage one)	0.5528 [0.5503, 0.5553]	0.3628 [0.3604, 0.3652]	0.3623 [0.3600, 0.3647]	0.4043 [0.4016, 0.4071]	0.4807 [0.4785, 0.4830]
two-stage re-ranker	0.7885 [0.7867, 0.7906]	0.5729 [0.5703, 0.5755]	0.5724 [0.5698, 0.5750]	0.6411 [0.6387, 0.6437]	0.6715 [0.6695, 0.6736]
fused + lifetime popularity*	0.5784 [0.5760, 0.5808]	0.3707 [0.3684, 0.3731]	0.3703 [0.3679, 0.3726]	0.4184 [0.4159, 0.4210]	0.4910 [0.4889, 0.4932]
<i>MIND small, val</i>					
BM25 (stage one)	0.5840 [0.5816, 0.5864]	0.3204 [0.3176, 0.3232]	0.2777 [0.2753, 0.2803]	0.2924 [0.2895, 0.2954]	0.3494 [0.3467, 0.3522]
embeddings (stage one)	0.6356 [0.6334, 0.6379]	0.3422 [0.3396, 0.3449]	0.2988 [0.2964, 0.3012]	0.3202 [0.3175, 0.3231]	0.3789 [0.3763, 0.3816]
fused (stage one)	0.6390 [0.6368, 0.6412]	0.3478 [0.3450, 0.3504]	0.3031 [0.3007, 0.3056]	0.3248 [0.3220, 0.3275]	0.3836 [0.3811, 0.3863]
two-stage re-ranker	0.6997 [0.6976, 0.7018]	0.3977 [0.3950, 0.4005]	0.3422 [0.3397, 0.3448]	0.3729 [0.3701, 0.3759]	0.4313 [0.4287, 0.4339]

Table 18: All accuracy metrics on val with 95% intervals.

G Reproducing the numbers

Every table in this note is generated from the results files by `docs/design-note/make_tables.py`; every measured number also has a row in `docs/FACTS.md` or `docs/ABLATIONS.md` with the command that produced it.

```
make venv && make download
python -m src.pipeline.split ebnerd_small          # one dataset per process
python -m src.retrieval.bm25 ebnerd_small
python -m src.retrieval.embeddings ebnerd_small
python -m src.retrieval.fuse ebnerd_small
python -m src.features.build ebnerd_small
python -m src.rerank.train ebnerd_small --tag final
python -m src.eval.run ebnerd_small --rerank-tag final
python -m src.eval.compare_reranks ebnerd_small ladder17 final
python -m src.eval.bench ebnerd_small --requests 2000 \
    --scale-points 0.1 0.25 0.5 1.0 --model-tag final
python -m src.eval.pool_size ebnerd_small
python -m pytest tests/
python -m src.submission.validate ebnerd submissions/ebnerd_predictions.zip
```

H Leaderboard screenshots

